Education Series #4: AN"OUT-OF-KILTER" NETWORK TUTORIAL

H. S. Swanson & R. E. D. Woolsey

Network flow algorithms have been developed to handle a special class of linear programming problems. These problems may each be described by a network whose links carry flow. Network algorithms take advantage of this special structure to produce an optimal solution much more quickly, with less storage required, and with virtually no round off error in comparison with general linear programming codes. The objective of this paper is to introduce the reader to the basic ideas involved in formulating a network problem, and to explain in a non-technical manner what the "out-of-kilter" network algorithm (1) does. This algorithm is very general and extremely efficient in solving network flow cost problems (as opposed to CPM, PERT, and GERT type networks).

Network problems, as you might suspect, can be thought of intuitively as networks (but don't of necessity physically represent such): points, places, or locations designated to be network "nodes" and links or "arcs" which connect certain of these nodes. These arcs allow movement between the nodes they connect. Movement or "flow" along each network arc is restricted by a maximum capacity (ability to handle flow per unit time) and a minimum required flow (perhaps greater than zero). Also known is the cost (assumed constant) of transmitting each unit of flow from one node to another along an arc. The flow is not required to be fluid in the sense of a liquid, but must meet a physical restriction which one might normally expect of a "flow". Roughly, flow cannot "stack up" at a node, or total flow into a node must equal total flow out of that node. This property is called conservation of flow.

It is often surprising to discover the wide variety of practical problems which may be posed as the analog of flow in a capacitated, cost network. Oil and gas pipeline, telephone lines, highway systems, electrical circuits, computer configurations, airline routes, the "flow" of work in an office, assembly lines, and in general most communication, transportation, and distribution systems form network structures. Problems as varied as determining optimal number and location of oil drilling platforms in the ocean, routing river flow to avoid flooding, and making the optimal assignment of men to machines are examples of (not-so-obvious) network flow problems. If a problem is to be solved using a network approach, it must be possible to relate the variables of the problem to the flow in arcs of visualized network and relate the costs or benefits accordingly.

In general, network problems may be characterized as falling into one of the three following categories:

1. find the maximum flow from a designated node to a different designated node subject to the capacity restrictions on flow in each arc and conservation of flow at each node;

2. find the maximum flow from a designated node to a different designated node which minimizes the cost incurred, subject to capacity restrictions on flow in each arc and conservation of flow at each node;

3. find that flow which minimizes the cost of shipping k units (k is a known integer which might be less than the maximum flow) from a designated node to a different designated node, subject to capacity restriction on flow in each arc and conservation of flow at each node.

The out-of-kilter algorithm can solve problems in each of the above categories (plus problems in some additional unmentioned categories) by proper selection of costs, capacities, and network configuration.

In order to discuss the algorithm, a small amount of jargon is necessary. A path from node A to node B in a network is a set of sequential arcs which forms a "route" from A to B. Arcs (1,2), (2,3), (3,4) form a path from node 1 to node 4 in the network of figure 1. A cycle in a network is a path which ultimately leads back to the starting node. Arc (4,1) in addition to the previously mentioned paths forms a cycle. A circulation in a network is a set of flows on the arcs of the network which preserves conservation of flow at each node. The flows of Table 1 define a circulation for the network of Figure 1 (note there are many other possible circulations). In the application that follows, it will be shown how to construct a circulation in a network.

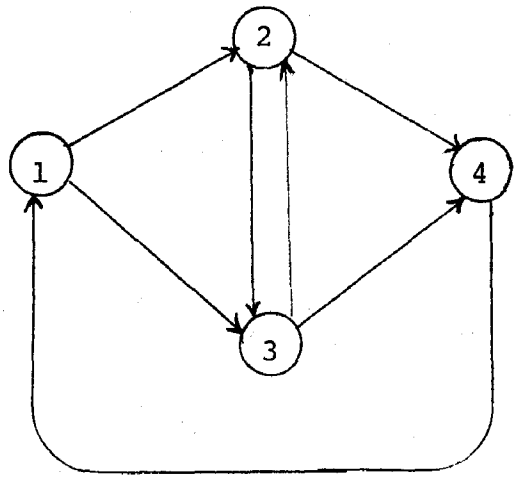



FIGURE 1

| ARC | FLOW |
|---|---|
| (1,2) | 1 |
| (1,3) | 1 |
| (2,3) | 0 |
| (2,4) | 1 |
| (3,2) | 0 |
| (3,4) | 1 |
| (4,1) | 2 |

TABLE 1

The out-of-kilter algorithm operates in such a way as to maintain a circulation in the network while re-routing flows so as to minimize cost and satisfy capacity restrictions on each arc. This generates an optimal solution to the problem if such a solution exists. The theoretical aspects of this process arise from the primal-dual theory of linear programming (complementary slackness) [1]. The brief discussion which follows is intended to give the reader an intuitive idea of how and why the algorithm works. A more detailed discussion may be found in earlier articles on this subject: [1], [2], [3].

To find the optimal flow in a network, the algorithm is initialized with a circulation flow of $\underline{0}$ in each arc (conservation is preserved at each node). Note that such a circulation might not satisfy capacity constraints (problem is dual feasible). In addition, a number must be assigned to each node (these "node numbers" are actually values of the dual variables, and they may be initialized at value $\underline{0}$). The "economic" or "intuitive" interpretation of each node number is the "value" or "price" of a unit of flow at that node. Examine an arc of the network, say from node s to node y. Since the cost $C_{xy}$ of flow on arc (x,y) is known, compute $D_{xy} = P_x + C_{xy} - P_y$, where $P_x$ ($P_y$) is the node number at node x (node y). Note that the economic interpretation of the number $D_{xy}$ is then the "value" of flow at node x plus the cost of moving from node x to node y along arc (x,y) minus the "value" of flow at node y. Then, intuitively, if

a. $D_{xy}<0$ the "value" of flow at node y is greater than the "value" of flow at node x plus the cost of getting the flow at node x to node y. This implies that the flow in arc (x,y) should be as great as possible.

b. $D_{xy}>0$, then similarly, the flow in arc (x,y) should be as little as possible (the cost of moving from node x to node y is greater than the increase in "value" of moving from node x to node y).

c. $D_{xy}=0$ the increase in "value" of moving from node x to node y exactly matches the cost of doing so; thus the flow in arc (x,y) need only satisfy the capacity restrictions on arc (x,y).

If the flow in arc (x,y) is "appropriate" for the value of $D_{xy}$, then arc (x,y) is said to be "in-kilter" (actually satisfied optimal conditions). Otherwise, an arc is said to be "out-of-kilter". If an arc (x,y) is out-of-kilter, the algorithm seeks to find a path from node y to node x which together with arc (x,y) forms a cycle along which the flow may be changed by a constant amount without disturbing conservation of flow at any node. Actually a special kind of path must be found -- one along which a change in flow would not worsen the "kilter condition" (a.,b.,c on the preceding page) of any arc. In addition, the out-of-kilter algorithm has a routine to change flows once a cycle is found for an out-of-kilter arc, and a routine to change the node numbers (dual variables) when that becomes necessary. When each arc is "in-kilter" the flow is optimal. To summarize, the algorithm proceeds as follows:

0. Start with any circulation and any set of node numbers.

1. Find an out-of-kilter arc, (s,t). If none, stop.

2. Determine if the flow in arc (s,t) is to be increased or decreased to bring (s,t) into kilter.
   If the flow should be increased, go to step 3.
   If the flow should be decreased, go to step 4.

3. Find a path from t to s along which the flow can be increased without causing any arc on the path to become "more" out-of-kilter. If a path is found, increase the flow along the path and also in (s,t). If (s,t) is in-kilter, go to step 1. If (s,t) is still out-of-kilter, repeat step 3. If no path can be found, go to step 5.

4. Find a path from s to t along which the flow can be increased without causing any arc to become more out-of-kilter. If a path is found, increase the flow along the path and decrease the flow in (s,t). If (s,t) is in-kilter, repeat step 4. If no path is found, go to step 5.

5. Change the node numbers. If (s,t) is in-kilter, go to step 1. If (s,t) is still out-of-kilter, repeat step 2. If the node numbers become infinite, stop. There is no feasible flow.

As an example of how to formulate a network problem, consider the frequently-occurring "assignment problem". Essentially this problem consists of trying to "match up" n men with n machines (or machines with jobs or men with jobs), in such a way as to maximize the efficiency of work. In general there are some men who cannot operate certain of the machines, but each man must be assigned to one and only one machine (do not assign more than one man to each machine).

The efficiency of each man on each (feasible) machine must be known. Usually the efficiency is measured by the average time required to perform a given task on a given machine (items/unit time, time to complete task, quality of output, etc.). Note that assigning each man to that machine which he operates with the greatest efficiency might not be a solution to the problem, since several men might be "most efficient" on the same machine. Those readers familiar with this problem should recognize that there are many variations. For example, there might be more men than machines, more machines than men, or one may wish to maximize the minimum efficiency of an assignment. Each of these problems may be solved by a simple adaption of the network formulation.

To solve the assignment problem described above, construct a network consisting of a node for each man (number these nodes 1 through n) and a node for each machine (numbered n+1 through 2n). The arcs of the network are determined by which men have the ability to operate which machines. In particular, if man i can operate machine j, then the network must have an arc from node i to node j. If n=3, the following network might result:

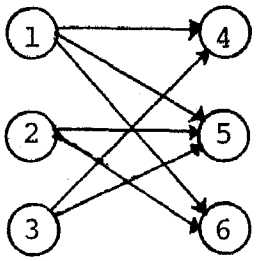



FIGURE 2

The network of Figure 2 shows that man 1 can operate each of machine 4,5, or 6; man 2 can operate machine 5 or 6; man 3 can operate machine 4 or 5. In order to preserve conservations of flow at each node and to prepare the network of Figure 2 for solution by the out-of-kilter algorithm, some additions must be made to the network. Add node S, node T, and the connecting arcs as shown in Figure 3. This reduces the network to a "single source - single sink" network and will simplify the conservation of flow problem. Node S is called the "super source" node (it generates all the available flow) and node T is called the "super sink" node (it eventually absorbs all the available flow). As a final addition to the network, connect node T to node S by an arc (T,S) called the "circulation arc". The circulation arc connecting the super sink to the super source provides for conservation of flow at each node of the network.

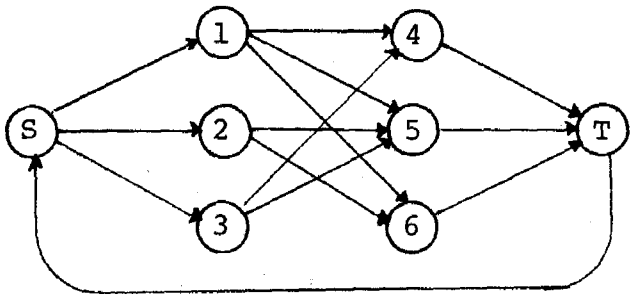



FIGURE 3

Now the upper bound, lower bound, and cost must be fixed for each arc in such a way as to insure that the assignment problem will be solved by the out-of-kilter algorithm operating on the network of Figure 3. Recall that the algorithm will seek to minimize the sum of cost times flow in each arc while satisfying upper and lower bounds on flow in each arc and maintaining conservation of flow at each node. Consider the ordered triples of Table 2 as values of the upper bound, lower bound, and per unit cost for the corresponding arc of the network shown in Figure 3. ($A_{14}$ is the efficiency of man 1 working on machine 4, etc.).

| ARC | (u,l,c) |
|---|---|
| (S,1) | (1,1,0) |
| (S,2) | (1,1,0) |
| (S,3) | (1,1,0) |
| (1,4) | (1,0,$-A_{14}$) |
| (1,5) | (1,0,$-A_{15}$) |
| (1,6) | (1,0,$-A_{16}$) |
| (2,5) | (1,0,$-A_{25}$) |
| (2,6) | (1,0,$-A_{26}$) |
| (3,4) | (1,0,$-A_{34}$) |
| (3,5) | (1,0,$-A_{35}$) |
| (4,T) | *1,1,0) |
| (5,T) | (1,1,0) |
| (6,T) | (1,1,0) |
| (T,S) | (3,3,0) |

TABLE 2

For the first three arcs of Table 2, the upper and lower bounds each have value one to insure that each man is assigned to exactly one machine. The following seven arcs have lower bound of zero so as not to require which man is assigned to which machine. The "cost" on each of these arcs is the negative of the man-machine efficiency, since the out-of-kilter is a minimizing algorithm (maximizing efficiency is equivalent to minimizing the negative of efficiency or "inefficiency"). The next three arcs have upper and lower

bounds each equal to one in order to insure that each machine is assigned exactly one man. The circulation arc insures that three assignments are made (this is insured regardless, due to conservation of flow requirements). Note that the only "costs" involved are the "inefficiency" measures, and these will be minimized.

Those arcs which have flow at optimality correspond to the optimal assignments to be made. Thus if there is flow in arc (1,6) at termination of the algorithm, man 1 should work on machine 6. As an added convenience, the flows will be in integer (counting) units so that no fractional assignments will be made. The network solution to the assignment problem (and especially solutions to larger, more realistic problems) is (are) usually obtained in 1/2 to 1/3 the time taken by other codes.

## REFERENCES


1. L. R. Ford and D. R. Fulkorson, Flows in Networks, Princeton University Press, New Jersey, 1962.

2. E. P. Durbin and D. M. Kroenke, "The Out-of-Kilter Algorithm: "A Primer", Rand Corporation Memorandum, RM-5472-PR, December, 1967.

3. D. R. Fulkerson, "An Out-of-Kilter Method for Minimal-Cost Flow Problems", J. Soc. Indust. Appl. Math., Vol. 9, 1961.